Does a shared AGENTS.md make coding-agent runs fail together? Coincident failures, voting and the reliability of agent-written changes

1. Background

In N-version programming, a vote over several versions of a program helps only if the versions rarely fail on the same inputs. Eckhardt and Lee showed that two versions drawn at random from one development process fail independently only if every input is equally likely to make a version fail [11]. Zheng, Okamura and Dohi asked whether generative AI can enhance the effectiveness of N-version programming [1]. They compared high-performing models (e.g., GPT-4o) with lower-performing ones, and varied the temperature and whether the conversation history was reused. Majority voting made the strong models more reliable but helped little with the weak ones. More diversity did not always mean higher reliability.

Two 2026 studies found that LLM-written versions fail together more often than independence predicts. Ron et al. had five agent harnesses implement the specification from the 1986 Knight and Leveson experiment. Two or more of the 48 accepted programs failed together on 429 of one million inputs, against about 115 expected under independence, and many of these failures traced back to parts of the specification that are hard or ambiguous. Voting over three programs still cut the mean failure count from 387 to 131 [5]. On 224 contest problems with twelve models, voting over three versions captured 0.43 of the gain that independent failures would allow, and 0.27 when all three came from one model [6].

Developers steer coding agents such as Claude Code and Codex with a context file, AGENTS.md or CLAUDE.md, that lists rules such as how to run the tests and where new code goes. On real Python tasks, files written by the developers raised task success by 2.4% on average (not significant, $p = 0.21$) and raised cost by up to 19%, although agents did use the tools a file named (uv 1.6 times per task when named, under 0.01 times when not) [8]. In a leave-one-out test of 18 generic rules, the only rules whose removal lowered the pass rate by more than 5 points were prohibitions, such as “do not refactor unrelated code” [9]. Neither study tests the code beyond the task’s own tests. Merged agent code also needs more fixes: in 182 repositories, bug-fixing commits changed 4.0% of agent-written lines within 180 days of merge, against 2.7% of human-written ones [10].

2. Purpose and research questions

Every run in a repository reads the same context file, so the file is part of the specification that all runs share. A rule can push every run toward the same design: if it tells the agent to use one helper and that helper mishandles an edge case, every run inherits the bug. Ron et al. [5] and Nogueira et al. [6] used no context file. As far as I know, no study has measured how often repeated agent runs on real tasks with a developer’s own context file fail together, or whether removing one real rule changes reliability beyond the tests.

RQ1. When several agent runs solve one real task with the same context file, how much more often do they fail on the same inputs than independence predicts, and which change between runs (file version, agent, model, temperature) lowers it most?

RQ2. Does replacing one real rule with neutral text change the reliability of the code, measured as the failure probability on the pull request’s tests and on generated inputs beyond them?

A checker confirms that each manipulation changed what the agent did. I report compliance but do not test it as an RQ.

3. Plan

Tasks and rules. The 138 tasks of [8], from 12 Python repositories with developer-written context files, are public with Docker images and test commands. I start from them and add newer merged pull requests from the same repositories, where possible merged after the models’ training cutoff. A task’s own tests fail before the change and pass after it. The agent starts from the code just before the merge, with the context file of that time, no git history, and a task text that does not mention the rule. I pick about 5 rules that a script can check and that change what code is written (such as which helper to use) rather than where files go, each with 2 tasks. A checker is kept only if it agrees with my hand labels on at least 90% of 20 or more examples, and I report Cohen’s kappa. As in [9], I screen about 20 candidate tasks with 3 runs per agent and drop tasks an agent never solves (27% in [9]). Unlike [9], I keep tasks it always solves, since I measure failures beyond the tests.

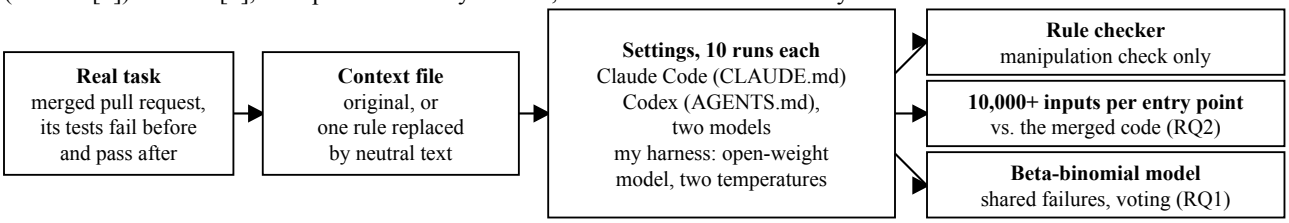


Fig. 1. Design. Only runs that pass the pull request’s tests are admitted as versions.

Settings and runs. Claude Code reads the file as CLAUDE.md and Codex as AGENTS.md, both pinned to fixed versions in a clean container; Codex also runs a second, smaller model to separate model from agent. The third agent is a small harness I will write in plain Python, as I built my desktop agent Yuki 2.0. It shows me the exact prompt the model receives and lets me set the temperature, which, as far as I know, neither CLI documents. It runs an open-weight model, Qwen3-30B-Coder as in [8], at two temperatures. That makes 820 runs: 400 main runs (10 tasks, 2 file versions, 2 agents, 10 runs), 100 for the second Codex model and 200 for the harness (both on the original file only), and 120 for screening. At the per-task costs in [8] (Claude Code with Sonnet 4.5 1.15 to 1.51 US dollars; Codex 0.32 to 0.57 with GPT-5.2, 0.18 to 0.22 with GPT-5.1 mini; Qwen3-30B 0.12 to 0.15), that is about 420 to 590 US dollars. Newer models may cost more: [9] reports about 5 US dollars per task and condition with Claude Opus 4.6. I would ask whether this fits a lab budget, and otherwise move more runs to the open-weight model.

Failures beyond the tests (RQ2). A run’s change is admitted as a version only if it passes the pull request’s tests, as in Knight and Leveson’s acceptance test and [5]. Inputs go to the entry points those tests call, so every admitted version implements them. The merged code is the reference [5], and two outputs count as the same under a fixed rule: a float tolerance (like realcompare in [5]), the same exception class, and order ignored for unordered collections. Failure probability is the share of failing inputs under a declared distribution: random inputs built from the type hints and the values in the pull request’s tests, at least 10,000

per entry point. I check 20 disagreements per task by hand and drop inputs where the merged code itself is wrong. RQ2 compares this probability and the test pass rate between the two file versions, with task as a random effect.

Boundary inputs. I also use the boundary coverage distance (BCD) method of Guo, Okamura and Dohi [2], but only as a stress test: with 30 to 50 inputs per program it cannot estimate a failure probability. In [2] the boundary points were found by hand; I take them from branch conditions in the merged code that compare a parameter with a constant. These are path-based boundaries, which [2] notes (Remark 2) differ from output-based ones; the pilot will report how many functions have them. I report the kill rate as in [2] (a mutant is killed when its execution path differs; here from coverage.py branch data) and by outputs. For strings and lists, which [2] leaves open, I will try one-element edits as minimal operations.

Shared failures (RQ1). For setting s, let $\theta_s(x)$ be the chance that a run fails on input x. Two runs both fail with probability $E[\theta_s(X)^2] = p_s^2 + \mathrm{Var}\,\theta_s(X)$, where p_s is the mean failure probability, so independence needs $\mathrm{Var}\,\theta_s = 0$ [11]. I model $\theta_s(X)$ with a Beta distribution, so the number of failing runs among k on one input is beta-binomial, and its intraclass correlation ρ_s measures dependence ($\rho_s = 0$ means independence). A 2-out-of-3 vote then fails with probability $E[3\theta^2 - 2\theta^3]$. I fit the model per task and setting by maximum likelihood. Because the same runs meet every input, I take intervals from a bootstrap that resamples runs. For runs from two settings a and b, the chance that both fail is $p_a p_b + \mathrm{Cov}(\theta_a, \theta_b)$, so I can ask which change between runs lowers the covariance most. The Knight and Leveson z of [5] and the pairwise score of [6] serve as baselines. The fitted model also gives the odd number of runs N that minimises $c_{\mathrm{run}}\, N + c_{\mathrm{fail}}\, \mathrm{P}(\text{majority of } N \text{ fails})$. In practice the vote would pick the patch whose outputs agree with most other runs, and the model predicts how often that pick is wrong.

Field data (if time permits). Agent code keeps arriving, so a finite-fault NHPP for one fixed code base does not fit. Instead, each merged agent change j (merge time τ_j, L_j added lines, covariates z_j) adds $\omega_j = L_j \exp(\beta' z_j)$ expected faults, found after a delay with distribution G, so $m(t) = \Sigma_j\, \omega_j\, G(t - \tau_j)$, with G from pysrat [4]. This is an infinite-server queue, which Dohi, Osaki and Trivedi used to unify finite and infinite fault NHPP models [12]. I would build fix data for about 20 AIDev repositories [7] from commit messages, as Qiu et al. did from GitHub issues [3]. Unlike the line-level Fine-Gray analysis of [10], this predicts how many fixes remain. Counts will be far below the 104 to 397 faults per dataset in [3], so I would pool repositories.

Limits. The results cover only Python and only functions whose inputs can be generated. Differential testing cannot see a bug that the merged code shares, which is exactly the shared-helper case above.

Table 1. Schedule

Period	Plan
Year 1, first half	Probability, statistics and reliability courses; beta-binomial and NHPP models; harness, checkers, inputs; pilot
Year 1, second half	Main runs; failures beyond the tests (RQ2); boundary inputs and mutation checks
Year 2, first half	Shared-failure model and voting (RQ1); first paper
Year 2, second half	Master's thesis; field data if time permits

4. Expected results

The main result will be a fitted model of how often agent runs fail together on real tasks and which change between runs lowers that most, so that a team can choose how many runs to vote over at a given cost. The second is whether removing one real rule changes reliability beyond the tests. I will release all tasks, code and data, and aim to publish at ISSRE or its AISQ workshop (co-chaired by Prof. Dohi in 2025), QRS or the Software Quality Journal.

5. My preparation and why this laboratory

I finished my B.E. in Computer Science and Engineering at Sri Sairam College of Engineering (VTU) in May 2026 with a CGPA of 8.0/10. So far my only statistics course is the third-semester Mathematics for Computer Science (BCS301), which covered probability distributions, Markov chains, hypothesis tests and ANOVA. A full statistics course is in my Year 1 plan. Since November 2025 I have been an AI engineer on the founding team at Layerpath. There I found that some rules in BrandMemory, which gives a company's writing rules to a model, had no effect: the brand voice section had the lowest priority and was the first cut by a 3,200-character limit. After the fix, one A/B comparison showed contractions falling from 6 to 0 and "you" and "your" from 15 to 1. One comparison is not a study, but since then I check whether a model actually follows an instruction. I use Claude Code and Codex every day. I review and test the changes they make, and I maintain the CLAUDE.md and AGENTS.md files they read. I also built Yuki 2.0 (public at github.com/mafex11/yuki-2.0), a desktop agent for Windows, in plain Python with the Anthropic SDK and no LangChain. I will leave Layerpath before I enrol and study full time.

The laboratory's own page pairs each technique with "conventional probabilistic evaluation" to show "quantitatively how much the quality actually improved", and my question needs that kind of answer. RQ1 takes the question of Prof. Zheng, Prof. Okamura and Prof. Dohi's study of N-version programming with generative AI [1] to real repository tasks, RQ2 uses the BCD method of [2], and the field study would use pysrat [4]. I can build and run the agents. I cannot yet fit and test reliability models on their failures, and I want to learn that in this laboratory.

References

[1] J. Zheng, H. Okamura, T. Dohi: Can Generative AI Enhance the Effectiveness of N-Version Programming?, SFPVV 2025, LNCS 16356, pp. 1-16, 2026.
[2] X. Guo, H. Okamura, T. Dohi: Optimal test case generation for boundary value analysis, Software Quality Journal 32(2), pp. 543-566, 2024.
[3] N. Qiu, T. Dohi, J. Wu, J. Zheng, H. Okamura: A Semi-Parametric Self-Exciting Software Reliability Model with Time-Varying Base Intensity and Shallow Neural Kernel, Int. J. Math. Eng. Manag. Sci. 11(5), pp. 1907-1928, 2026.
[4] H. Okamura: pysrat, a Python package for software reliability analysis and modeling, version 0.2.7, 2026, https://github.com/SwReliab/pysrat.
[5] J. Ron, B. Baudry, M. Monperrus: N-Version Programming with Coding Agents, arXiv:2606.20158, 2026.
[6] R. P. Nogueira et al.: A Systematic Methodology for Evaluating Failure Independence in LLM-Generated Code, arXiv:2607.02808, 2026.
[7] H. Li, H. Zhang, A. E. Hassan: AIDev: Studying AI Coding Agents on GitHub, MSR 2026.
[8] T. Gloaguen et al.: Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?, arXiv:2602.11988, 2026.
[9] X. Zhang et al.: Guardrails Beat Guidance: A Large-Scale Study of Rules, Skills, and Persistent Configuration for Coding Agents, arXiv:2604.11088, 2026.
[10] C. S. Xia, C. Miller: Do These Violent Delights Have Violent Ends? Measuring the Post-Merge Fate of Agentic Code, arXiv:2607.09902, 2026.
[11] D. E. Eckhardt, L. D. Lee: A Theoretical Basis for the Analysis of Multiversion Software Subject to Coincident Errors, IEEE TSE 11(12), pp. 1511-1517, 1985.
[12] T. Dohi, S. Osaki, K. S. Trivedi: An Infinite Server Queueing Approach for Describing Software Reliability Growth, APSEC 2004, pp. 110-119.